

Kravis Leadership Institute
Leadership Research Fellowship

The NP Condition



KRAVIS LEADERSHIP
INSTITUTE

CLAREMONT MCKENNA COLLEGE

Claire Vlasses
Professor Gabbrielle Johnson
April 2025

Table of Contents

Introduction	3
On Computability	4
On Complexity	5
Measuring Computational Efficiency	5
Complexity Classes	8
Class P	8
Class NP	9
The P vs NP Debate	10
Complexity Applied	11
Can a Computer Think?	12
The Mind, Computationally	13
The Complexity of Cognition	15
Reflex Actions and Basic Arithmetic: P Complexity	16
Logical Reasoning: NP-Complete Complexity	17
The NP Condition	17
Heuristics Over Optimization	18
The Complexity of Leadership	18
Extending the NP Condition	20
Conclusion	21
References	22

Introduction

“But for short conversations, a man may carry implements in his pockets, and under his arms, enough to supply him; and in his house, he cannot be at a loss. Therefore the room where company meet who practise this art, is full of all things, ready at hand, requisite to furnish matter for this kind of artificial converse.”

— Gulliver’s Travels 1726, III.V

People have long been tempted to believe that thought could be mechanized; that intelligence could be broken down, stored, and carried in our pockets. Today, artificial intelligence is often discussed in similar terms, as if intelligence itself could be packaged, replicated, and deployed on command.

However, as identified in Gulliver’s Travels, centuries before the advent of the modern computer, the simple retrieval of preassembled parts is artificial, not genuine, intelligence. Human intelligence is a complex and dynamic creation. It is exactly this *complexity* that separates human from machine.

By examining the types of problems computers can and cannot solve efficiently, we can gain new insights into the challenges people face when we think, decide, and lead. Computational theory thus offers a lens through which to understand the structure of intelligent behavior.

When taken seriously, existing models from computability theory, complexity theory, and the computational theory of mind suggest a broader framework for understanding cognition, decision-making, and leadership in complex systems. These models distinguish not only what is possible in principle, but what is feasible in practice.

This paper begins by introducing relevant background in mathematical logic and theoretical computer science, with particular attention to computability and complexity. I then apply these concepts to human cognition, focusing on the use of heuristics and the role of bounded rationality in both individuals and organizations. From there, I extend this argument into the domain of leadership, showing how complexity theory helps explain the limits of centralized control and the need for distributed, adaptive processes.

These threads build toward what I call the *Nondeterministic Polynomial-time (NP) Condition*—the structural fact that human life is lived under asymmetry: between recognition and production; evaluation and construction; knowing what is right and knowing how to get there.

On Computability

To understand the NP Condition, we will first need to understand how computers solve complex problems. In the 1920s, there was a movement in mathematics to codify what it means for a problem to be solvable, or, *computable*. The drive to codify what is computable was motivated by an open challenge issued by mathematician David Hilbert, who asked if there was a way that all of mathematics could be captured by a specific set of rules (axioms). Calling it the *Entscheidungsproblem*, it was a challenge to determine whether there exists a general algorithmic procedure capable of deciding whether any mathematical statement formulated in first-order logic was true or false.¹ Hilbert was asking for what would now be called a “decision procedure” for all of mathematics.² The idea was that if mathematics was fully axiomized, then some mechanical process—an algorithm—could take as input any precise mathematical statement, and, after a finite number of steps, decide whether the statement was true or false. If the Entscheidungsproblem was indeed proved possible, it would imply that all mathematical reasoning could, in principle, be automated.³

In response to the Entscheidungsproblem, Kurt Gödel presented his Incompleteness Theorems, posing a fundamental challenge to Hilbert’s vision. Gödel proved that for any sufficiently powerful formal system:

1. There are true mathematical statements that are unprovable within the system.
2. The system’s consistency cannot be proven from within the system itself.⁴

Gödel’s work suggests that even if there was a complete list of mathematical axioms, there would still be statements whose truth could never be determined algorithmically.⁵ That is, there is no reasonable list of axioms from which we can prove exactly all true mathematical statements.

A few years later, Alonzo Church and Alan Turing independently proved that the Entscheidungsproblem is unsolvable. Turing’s approach was particularly influential because his

¹ First-order logic is a collection of formal systems that uses quantifiers, variables, and relations. For example, rather than propositions like “all puppies are cute,” in first-order logic, one can have expressions in the form of “for all x, if x is a puppy, then x is cute.”

² Neil Immerman, “Computability and Complexity,” in *The Stanford Encyclopedia of Philosophy*, Winter 2021 Edition, ed. Edward N. Zalta.

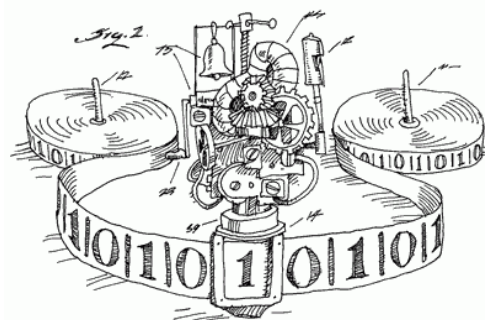
³ Immerman, “Computability and Complexity.”

⁴ Such as Peano Arithmetic (one of my personal favorites!)

⁵ Panu Raatikainen, “Gödel’s Incompleteness Theorems,” in *The Stanford Encyclopedia of Philosophy*, Spring 2022 Edition, ed. Edward N. Zalta.

machine model, the Turing Machine, provides a mechanistic, step-by-step process for defining computation. A Turing machine consists of:

1. An infinite tape divided into discrete cells, each capable of holding a symbol.
2. A read/write head that moves left or right along the tape.
3. A finite set of states that determine the machine's behavior based on its current state and the symbol it reads.⁶



By showing what was not computable, Church and Turing were able to show what was: Now recognized as the Church-Turing Thesis, a mathematical problem is *computable* if it can be carried out in a finite number of steps on a Turing machine.⁷ A consequence of the Church-Turing Thesis is that all reasonable models of computation, including lambda calculus, recursive function theory, and modern programming languages are equivalent in terms of what they can compute. In other words, any function computable by one model can be computed by any other.⁸ This convergence suggests that the concept of computability is not merely a technical detail, but a fundamental feature of logic and information processing.

On Complexity

While the Church-Turing Thesis defines the boundaries of what is theoretically computable, understanding the practical structure of problems requires a deeper classification: whether a computation can be feasibly carried out. If computability theory asks what problems are solvable in principle, computational complexity theory asks what problems are solvable *in practice*.

Measuring Computational Efficiency

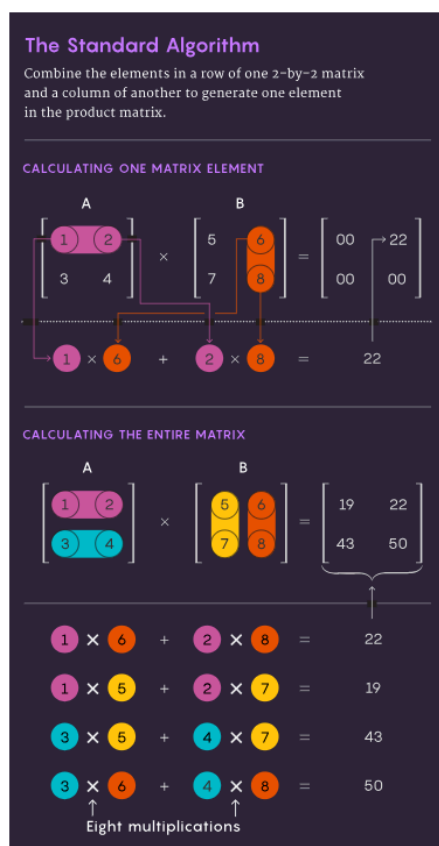
The efficiency of an algorithm is measured in terms of time and space complexity, which quantify the amount of time and memory required to solve a problem as a function of input size

⁶ Alan M. Turing, "On Computable Numbers, with an Application to the Entscheidungsproblem," *Proceedings of the London Mathematical Society* 42, no. 1 (1936): 230–265, <https://doi.org/10.1112/plms/s2-42.1.230>.

⁷ Immerman, "Computability and Complexity."

⁸ Turing machines, and the more generalized Universal Turing Machine that followed, provide a universal model of computation.

n .⁹ For some given input n , the time complexity of an algorithm refers to the number of basic computational steps required for an algorithm to solve a problem of size n . One might think that, once we know something is computable, whether it takes 10 seconds or 20 seconds to compute is obviously the concern of engineers rather than philosophers. However, that conclusion would not be so obvious if the question were one of 10 seconds versus 10^{10} seconds! As an intuitive example, consider the difference between the number of steps you might take walking down one city block compared with the steps you would take to walk down every possible city block, in every city, in every country. The differences between the running time of certain algorithms can be so vast that they must be treated as qualitatively distinct.¹⁰



Computer scientists use Big- O notation to compare the complexity of different algorithms. Big- O notation classifies the asymptotic upper bound on an algorithm's running time as a function of the input size. An algorithm is considered efficient if its running time can be upper-bounded by a polynomial function of n , and inefficient if it grows exponentially with n .¹¹

To demonstrate the practical importance of time complexity, consider matrix multiplication, a fundamental operation used in everything from sharpening computer graphics to optimizing logistical networks.

The traditional way of multiplying two n -by- n matrices, by multiplying numbers from each row in the first matrix by numbers in the columns in the second, requires n^3 separate multiplications. For 2-by-2 matrices, that means 2^3 or 8 multiplications.

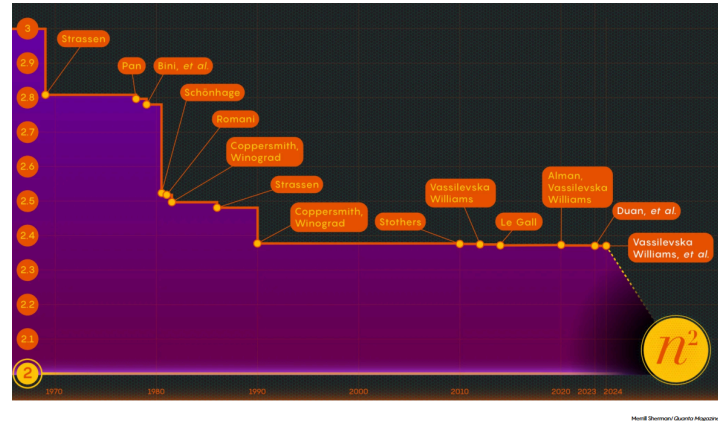
⁹ For simplicity, I chose to focus specifically on time complexity in this paper, although there are other complexity constraints too.

¹⁰ Scott Aaronson, "Why Philosophers Should Care About Computational Complexity," *MIT Press* (2011), <https://doi.org/10.48550/arXiv.1108.1791>.

¹¹ There are obvious objections to this—an algorithm that takes 1.1^n steps would be faster than an algorithm that takes $n^{100000000}$ steps. But, in practice, polynomial-time algorithms are so often efficient and exponential functions are often so inefficient, that complexity theorists consistently make this identification.

In 1969, mathematician Volker Strassen discovered an algorithm that could multiply 2-by-2 matrices in just seven multiplications and 18 additions. Because multiplications scale relative to input size, and additions remain constant no matter the input size, this finding reduced the overall Big- O running time from $O(n^3)$ to $O(n^{2.81})$. As recently as 2024, researchers managed to lower the running time to be $O(n^{2.376})$.¹²

This improvement might seem minuscule, but the consequences are profound. Imagine a computer capable of performing 10^3 (1,000) operations per second. At n^3 complexity, completing this computation would take 31.69 years. At $n^{2.81}$, the same task would take 8.529 years. At $n^{2.376}$ time,



the program would run for 5.106 months. If the program were to run in n^2 time, it would take 11.574 days; in $n^{1.5}$ time, 8.784 hours; and in linear time, n , it would take just 16 minutes.

The difference becomes even more noticeable at larger scales. The modern 4-core CPU can do floating point multiplications at 200 Billion operations per second. Even at that rate, performing an n^3 time computation would take more than 2,535,047,025,122,316,018,962,151 years.

Clearly, the time complexity of an algorithm is more than a theoretical concern; it fundamentally determines what is computationally feasible. Even matrix multiplication, which is still polynomial and thus technically feasible, shows how vital time complexity is to real-world computation.

¹² Nadis, Steve. "New Breakthrough Brings Matrix Multiplication Closer to Ideal." *Quanta Magazine*, Simons Foundation, 7 Mar. 2024, www.quantamagazine.org/new-breakthrough-brings-matrix-multiplication-closer-to-ideal-20240307/.

Complexity Classes

A central goal of computational complexity theory is to classify problems into different classes based on their efficiency. The classifications of complexity classes have been clearly, elegantly, and precisely drawn.¹³ To develop the foundation of the NP Condition, I focus on P and NP.¹⁴

Class P

Mathematicians Alan Cobham and Jack Edmonds identified the complexity class, P, of problems recognizable in some polynomial amount of time, as being an excellent mathematical wrapper of the class of feasible problems.¹⁵ More formally, the class P consists of all decision (yes/no) problems that can be efficiently solved by a deterministic Turing machine in polynomial time.¹⁶

An algorithm has polynomial time complexity if its running time is $O(n^k)$ for some non-negative integer k . In practice, mathematicians assume that problems in P are, at least in theory, tractable.¹⁷

There are many problems that fall into the P class. Consider, for example, the standard way to multiply two integers. We often start right from left, multiplying each digit and scaling the process as we move.¹⁸ If the integer has n digits, we have to complete n multiplications, which gives us $O(n)$ time steps. Since we do this for the other number too, our total running time for multiplying two integers is $O(n^2)$. The running time of this process scales with the size of the input n , placing it firmly in the P class.¹⁹



¹³ Immerman, “Computability and Complexity.”

¹⁴ Some other complexity classes include PSPACE, which consists of those problems solvable using some polynomial amount of memory space, and EXPTIME, the set of problems solvable in time $2^{p(n)}$, for some polynomial p . Some problems, like the Halting Problem, are undecidable and lie outside all these classes.

¹⁵ Immerman, “Computability and Complexity.”

¹⁶ A deterministic Turing machine means that it accepts a problem instance and a valid solution.

¹⁷ Examples of problems in P:

- Sorting a list of numbers: $O(n \log n)$.
- Determining whether a number is prime via AKS primality test, $O(n^6)$.
- Finding the shortest path in a graph seen with Dijkstra’s algorithm, $O(n^2)$.

¹⁸ “Multiplication: Definition, Symbol, Table,” *GeeksforGeeks*, Sanchhaya Education Private Limited, last modified April 16, 2025, <https://www.geeksforgeeks.org/multiplication/>.

¹⁹ As another example, imagine you have a To-Do list. In order to check if you wrote something, say “Eat breakfast” down on your list, you’ll need to quickly scan and look to see if it is there. If you are starting at the top of the list, and “Eat breakfast” is the first thing at the top of the list, the time complexity would simply be constant— $O(1)$. But, if “Eat breakfast” is at the very bottom of your list, you will need to look through the whole thing before

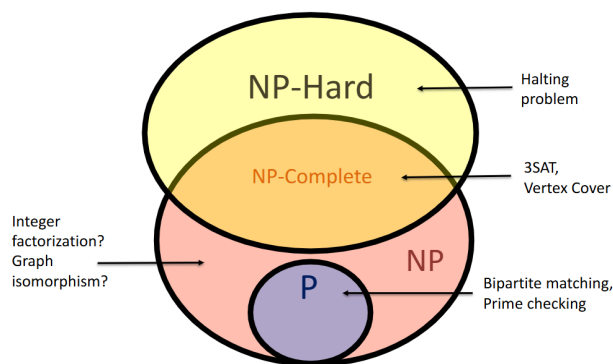
Class NP

The class NP, coined Nondeterministic Polynomial Time, consists of decision problems for which a given solution can be efficiently *verified* in polynomial time.

To be verified means that even if a problem is hard to solve, checking whether a given solution is correct is not hard.²⁰ For these problems, no polynomial-time algorithm is known to find solutions efficiently, but if a candidate solution is given, mathematicians can verify its correctness in polynomial time. For NP problems, the possible solution space explodes combinatorially.

For instance, consider Sudoku puzzles or jigsaw puzzles. Finding a correct configuration is meaningfully more difficult than verifying a completed Sudoku grid or puzzle assembly is correct. Solving a Sudoku puzzle requires checking many possible number placements. The number of combinations increases drastically as the grid size grows. The cost of “solving” quickly becomes prohibitive.

An algorithm that does not have a known polynomial-time algorithm for an optimal solution is classified as an NP-Hard problem. Exponential running time algorithms fall under this distinction. A problem has exponential complexity if its resource usage can be expressed as an exponential function of the input size, typically $O(2^n)$ or $O(c^n)$ for some constant $c > 1$. Exponential functions grow much faster than polynomial functions. Even for relatively small input sizes, the resource



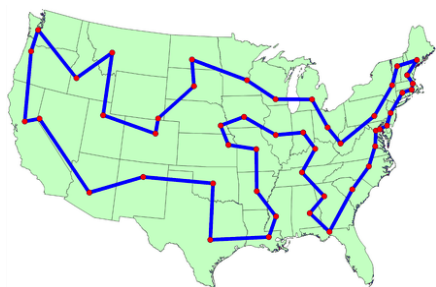
finding it. If there are n items on the list, then the time complexity would be $O(n)$. Because we look at the worst-case when identifying the Big- O running time, this “algorithm” would be assigned $O(n)$ complexity, because we cannot rely on “Eat breakfast” being at the start of the list. This linear search algorithm is still polynomial in complexity, in that $O(n) = O(n^1)$, and is thus considered efficient.

²⁰ Examples of Problems in NP

- Boolean Satisfiability (SAT): Given a Boolean formula, is there an assignment of true/false values that makes it true?
- Traveling Salesman Problem (TSP): Given a set of cities and distances between them, is there a tour of length $\leq k$? What is the shortest route that visits a set of cities once and returns to the origin? This arises in everything from delivery routing to DNA sequencing.
- Integer Factorization: Given an integer n , find its prime factors.
- Knapsack problem: Given a set of items with weights and values, what subset maximizes value without exceeding the weight limit? This models resource allocation, budgeting, and even moral trade-offs.

requirements can become impractically large. Due to the rapid growth in resource requirements, these algorithms often become infeasible as the input size increases. Algorithms with exponential complexity are generally considered intractable for large input sizes.²¹

A classic example is the Travelling Salesman Problem (TSP), where a salesman needs to find the shortest route to visit a set of cities, starting and ending at the same city, while visiting each city exactly once.²² The goal is to minimize the total distance or cost of the trip. As the number of cities increases, the time required to find the optimal solution grows exponentially, giving the running time $O(n!)$. This suggests that finding the absolute shortest route is computationally very expensive, especially for large numbers of cities.



In practice, algorithms for NP problems like TSP use heuristics and approximate solutions to cut down running time costs. A familiar example is the use of navigation apps on a smartphone. When a user requests directions, the system does not guarantee the absolute shortest path. Instead, it quickly produces a route that is reasonable and feasible. Underlying this is a heuristic algorithm, designed to provide a good-enough answer within practical time constraints rather than exhaustively computing the optimal solution.

The P vs NP Debate

As we've already seen, Hilbert's question – *could there exist an algorithm that determines the truth or falsehood of any well-formed mathematical statement?* – was proved impossible by the work of Gödel, Church, and Turing.

However, in a now-famous 1956 letter to John von Neumann, rediscovered decades later, Gödel proposed a new framing: Instead of asking whether proofs can be found algorithmically, mathematicians should ask whether they can be *efficiently* found. Specifically, given a formal system F , could one determine whether a mathematical statement S has a proof in F with n or fewer steps?

²¹“Difference Between Exponential and Polynomial Complexities,” *GeeksforGeeks*, Sanchhaya Education Private Limited, last modified August 6, 2024, <https://www.geeksforgeeks.org/difference-between-exponential-and-polynomial-complexities/>.

²² Jessica Yu, “Traveling Salesman Problem,” *Computational Optimization Open Textbook – Optimization Wiki*, Cornell University, Spring 2014, https://optimization.cbe.cornell.edu/index.php?title=Traveling_salesman_problem.

This problem is technically decidable: By exhaustive search, one could always check all 2^n symbol sequences of length n to see if any form a valid proof. However, this brute-force approach is computationally infeasible. Even for relatively small values of n , the number of possibilities grows exponentially, quickly exceeding the computational capacity of the known universe. The crucial question, as Gödel suggested, is whether a vastly more efficient approach exists, one that avoids exhaustive search and instead finds proofs in polynomial time:

“If there actually were a machine with [running time] $\sim Kn$ (or even only with $\sim Kn^2$) [for some constant K independent of n], this would have consequences of the greatest magnitude. That is to say, it would clearly indicate that, despite the unsolvability of the Entscheidungsproblem, the mental effort of the mathematician in the case of yes-or-no questions could be completely replaced by machines. One would indeed have to simply select an n so large that, if the machine yields no result, there would then also be no reason to think further about the problem.”²³

Thus, the essential issue is not whether problems are theoretically decidable, but whether they are feasibly solvable. Despite the theoretical unsolvability of the Entscheidungsproblem, if efficient algorithms existed, machines could effectively handle all meaningful cases. In practice, this would collapse the distinction between what is formally unsolvable and what is practically solvable. This distinction leads directly to the central open question of computational complexity: the P versus NP problem.

$$P \stackrel{?}{=} NP$$

If P were equal to NP, then every problem whose solution could be quickly verified could also be quickly solved. The existence of a machine with running time $\sim Kn$ or $\sim Kn^2$, would imply not merely theoretical computability, but practical tractability across decision problems. While no one has yet proven whether $P = NP$ or $P \neq NP$, most complexity theorists strongly suspect that the latter is true. If $P \neq NP$, then verification does not imply discovery: knowing a solution is correct does not entail that one can efficiently find it. However, if $P = NP$, it would mean that finding mathematical proofs is no harder than checking them—essentially, mechanizing much of human discovery.²⁴

Complexity Applied

The polynomial and exponential time distinction also plays an important role in philosophy. Consider the largest known prime number currently identified by the Great Internet Mersenne Prime Search (GIMPS):

²³ Michael Sipser, “The History and Status of the P Versus NP Question,” in *Proceedings of the ACM Symposium on Theory of Computing* (New York: ACM, 1992), 603–618.

²⁴ Aaronson, “Why Philosophers Should Care,” 7.

$$p = 2^{136,279,841} - 1$$

Mathematicians regard p as a “known prime,” even though its decimal expansion is too large to write down (it would require over 30,000 pages).²⁵ But why do mathematicians consider this number “known,” while they do not apply the same designation to the first prime greater than p ?

The reason lies in computational efficiency. The primality of p can be verified in polynomial time using efficient algorithms.²⁶ In contrast, finding the next prime lacks a known polynomial-time algorithm.²⁷ This distinction reveals that *knowledge* itself is tied to computational feasibility. If we define knowledge in terms of computational tractability, then some numbers are “known” in a deeper sense than others, not because of their mathematical existence, but because humans can compute them efficiently.

The world of complexity theory is one of scarcity—where time, memory, attention, and energy are constrained. It is a theory about the conditions of constraint. In computational complexity theory, the question is not merely “Can this problem be solved?” but rather “Can it be solved feasibly by the kind of agent we are, within the resources we have, under the pressures we face?” This reframes computation from a Platonic ideal to a real-world activity which is embedded, embodied, and bounded.

Can a Computer Think?

The relationship between computability and complexity naturally invites deeper inquiry. If knowledge is tied to complexity, then perhaps cognition can be understood within those same bounds. If the processes we call thought can be modeled as computations, could it be simulated by a sufficiently powerful computer in terms of complexity?

If thought processes are modeled as computations, their feasibility depends not only on whether they are computable in principle, but whether they can be executed within realistic complexity constraints. Simulating cognition, then, is a question of tractability.

²⁵“2136279841-1 Is the New Largest Known Prime Number,” *Great Internet Mersenne Prime Search*, Mersenne Research, Inc., last modified October 21, 2024, <https://www.mersenne.org/>.

²⁶ Algorithms such as the Lucas-Lehmer test.

²⁷ Aaronson, “Why Philosophers Should Care,” 10.

The Mind, Computationally

The Computational Theory of Mind (CTM) is a foundational view in cognitive science and philosophy of mind that proposes that mental processes are forms of computation; an execution of algorithms on mental representations.²⁸ Under this theory, the brain functions as an information-processing machine. Mental states correspond to symbolic representations and thought processes correspond to rule-governed manipulations of these representations. Thoughts, beliefs, and perceptions are considered analogous to data structures in a computer, manipulated through systematic operations. If thinking is computational, then in principle, intelligence can be replicated on a sufficiently powerful machine.

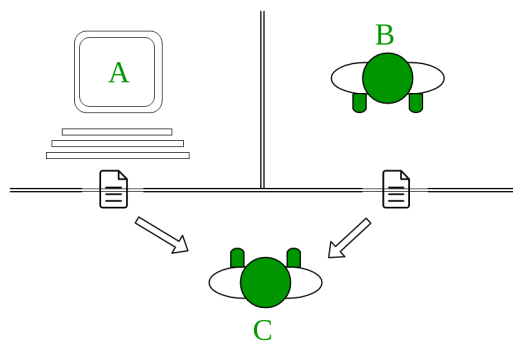
CTM rests on several key assumptions about how cognition functions as a computational process. Mental representations form the foundation of thought, positing that thoughts, concepts, and perceptions exist as discrete symbolic structures physically realized in the brain. These representations serve as the building blocks for cognitive processes, allowing the mind to store, manipulate, and retrieve information in a structured manner.

Another core assumption is that cognitive functions operate through algorithmic processing. Just as a computer executes a program by applying formal rules to inputs, the brain systematically transforms mental representations according to logical, rule-based procedures. Logical reasoning, problem-solving, and learning are modeled as structured computational operations within this framework.

Some philosophers insist that computers, no matter how sophisticated they become, will at best mimic rather than replicate thought.²⁹ Turing anticipated these worries.³⁰ He proposed a test, the Turing Test, to shift the focus from a hopelessly vague question of “Could a computer think?” to:

*Can a machine, in conversation, convince a human interrogator that it is also human?*³¹

The test involves a human judge engaging in a text-based conversation with a human and a machine. If the judge cannot reliably distinguish between the two, the machine is said to have passed the test,



²⁸ Michael Rescorla, “The Computational Theory of Mind,” in *The Stanford Encyclopedia of Philosophy*, Winter 2024 Edition, eds. Edward N. Zalta and Uri Nodelman.

²⁹ Rescorla, “The Computational Theory of Mind.”

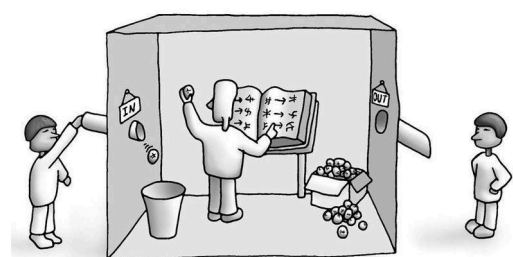
³⁰ A. M. Turing, “Computing Machinery and Intelligence,” *Mind* 59, no. 236 (1950): 433–460.

³¹ Turing, “Computing Machinery and Intelligence.”

suggesting that it exhibits intelligent behavior.³² There are numerous programs that have passed variants of the Turing test, like OpenAI's GPT-4.5, although some maintain the original criterion has never been met.³³

Critics of the test argue that it is insufficient evidence of genuine intelligence. A machine may imitate human emotions, but does it *feel* love, fear, or joy? It does not age, experience pain, or possess a self-awareness in the way that humans do.³⁴

In 1980, philosopher John Searle put forth one of the best known arguments challenging the assumption that successfully passing the Turing Test necessarily implies real intelligence. Searle presents the *Chinese Room Argument*, a thought experiment in which a



person inside a sealed room manipulates Chinese symbols according to syntactic rules without any actual comprehension of their meaning. Even if the outputs are indistinguishable from those of a native speaker, the system itself lacks semantic understanding—it processes symbols mechanically, without grasping their significance.³⁵ Searle contends that no purely

computational system can achieve understanding solely through symbol manipulation. This argument suggests that a machine could, in principle, simulate intelligent behavior without possessing any genuine cognition or self-awareness.³⁶

One challenge to Searle's argument, however, uses complexity theory to argue that it oversimplifies the dynamic nature of intelligence. To illustrate, consider an interaction between two people, George and Lisa. Within moments of conversation, George recognizes Lisa as

³² "Turing Test," *GeeksforGeeks*, Sanchhaya Education Private Limited, last modified September 16, 2024, <https://www.geeksforgeeks.org/turing-test-artificial-intelligence/>.

³³ Zena Assaad, "ChatGPT Just Passed the Turing Test. But That Doesn't Mean AI Is Now as Smart as Humans," *The Conversation*, April 8, 2025, <https://theconversation.com/chatgpt-just-passed-the-turing-test-but-that-doesnt-mean-ai-is-now-as-smart-as-humans-253946>.

³⁴ Aaronson, "Why Philosophers Should Care," 11.

³⁵ Rachel Anne Williams, "Searle's Chinese Room Thought Experiment: A Twist," *Medium*, April 15, 2018, <https://medium.com/@transphilosophr/searles-chinese-room-thought-experiment-a-twist-c7eb28f65e6c>.

³⁶ David Cole, "The Chinese Room Argument," in *The Stanford Encyclopedia of Philosophy*, Winter 2024 Edition, eds. Edward N. Zalta and Uri Nodelman.

another human. This suggests that there is a finite upper bound on the bits of information exchanged before George reaches this conclusion.³⁷

Now, imagine a lookup table T that stores every possible sequence of interactions between George and Lisa. This table records all potential conversational histories and dictates Lisa's next response based on prior exchanges, following a function $f_{Lisa}(T)$. Since the length of a human conversation is finite, this table would, in theory, also be finite and computable.³⁸

However, the complexity of such a look-up table is an important consideration. The number of possible conversation paths grows exponentially with the length of the dialogue. Consequently, storing and accessing the required data would demand an exponential increase in memory, time, or processing power, making this approach computationally infeasible (and likely a table bigger than the entire universe).

Searle's analogy would essentially require one enormous table that stores every possible input-response pair. He is right in believing that a Turing Test-passing lookup-table would lack sentience, consciousness, or self-awareness, as it merely retrieves precomputed responses. If a computer program were actually *intelligent*, it could not rely on brute-force computation alone.³⁹

This raises a critical question: Does *intelligence itself* require computational efficiency?

If a program required exponential time or memory resources to "think," its intelligence would be theoretical rather than practical, demanding more computational power than the universe could provide. However, a computer system constrained by polynomial complexity could, in principle, exhibit true intelligence if it dynamically computes responses rather than storing them. In this case, it would be intelligent precisely *because* it operates within polynomial bounds. If this reasoning holds, then the distinction between polynomial and exponential complexity has metaphysical significance.

The Complexity of Cognition

An efficient computer program must operate within reasonable time, memory, and computational constraints. If cognition can be fully mechanized, it would have to simulate the complex activity of the neurons and synapses in the human brain.

³⁷ Raphael Bousso, "Positive Vacuum Energy and the N-Bound," *Journal of High Energy Physics* 2000, no. 11 (2000): 038, <https://arxiv.org/abs/hep-th/0010252>.

³⁸ Aaronson, "Why Philosophers Should Care," 12.

³⁹ Aaronson, "Why Philosophers Should Care," 12.

Human brains operate with approximately 10^{10} neurons and 10^{14} synapses, with complex, parallel computations occurring continuously.⁴⁰ Simulating this level of complexity on a digital computer presents significant challenges:

1. **Memory Requirements:** If each neuron's state requires only a few bits of storage, a naive simulation would require terabytes of RAM just to store neural activity.
2. **Processing Power:** If each synaptic update requires an arithmetic operation, a brain simulation might require exascale computing power (beyond the capabilities of modern supercomputers).
3. **Time Complexity:** Even if a neural simulation is feasible, would it run in *real-time*? Or would it take exponential time, making it computationally infeasible?

However, the brain exhibits several computational optimizations that dramatically reduce the complexity of its operations:

- **Sparse Representations:** Only a fraction of neurons are active at any given time, reducing the effective computational load.
- **Parallel Processing:** Unlike classical computers, which execute instructions sequentially, the brain processes information in a highly parallelized manner.
- **Energy Efficiency:** The brain consumes only about 20 watts of power, far more efficient than any artificial neural network running on silicon-based hardware.⁴¹

If human intelligence is computational, then cognitive tasks should be analyzable in terms of algorithmic complexity. Many cognitive abilities, such as vision, reasoning, and language processing, map onto well-studied computational problems, some of which are known to be NP-complete or even PSPACE-complete.⁴²

Reflex Actions and Basic Arithmetic: P Complexity

Some of the simplest computations performed by biological systems occur at the level of reflex actions. Reflexes, such as pulling your hand away from a hot stove, occur without conscious

⁴⁰ Interestingly, despite the fact that brains have so many more synapses than computers have transistors, most modern multi-core computers are only 100 times slower than the brain, because of a computer's multi-gigahertz processor speed: Naveen Nagarajan and Charles F. Stevens, "How Does the Speed of Thought Compare for Brains and Digital Computers?," *Current Biology* 18, no. 17 (2008): R756–R758, <https://doi.org/10.1016/j.cub.2008.06.043>.

⁴¹ Peter J. Gebicke-Haerter, "The Computational Power of the Human Brain," *Frontiers in Cellular Neuroscience* 17 (August 6, 2023), <https://doi.org/10.3389/fncel.2023.1220030>.

⁴² Some researchers speculate that quantum processes might play a role in cognition, though this remains controversial: Stuart Hameroff and Roger Penrose, "Consciousness in the Universe: A Review of the 'Orch OR' Theory," *Physics of Life Reviews* 11, no. 1 (2014): 39–78, <https://doi.org/10.1016/j.plrev.2013.08.002>.

deliberation, relying instead on hardwired neural pathways in the spinal cord. These actions happen in constant time, meaning that their execution is independent of the complexity of the input. In computational terms, reflex actions exhibit $O(1)$ complexity.

In contrast, basic arithmetic operations, such as adding two numbers, require more processing but remain computationally feasible. The addition of two n -bit numbers requires examining each digit sequentially, leading to an $O(n)$ time complexity. Although linear scaling becomes significant for very large numbers, humans have developed cognitive shortcuts and pattern-based strategies to perform arithmetic efficiently, often breaking problems into smaller components.

Logical Reasoning: NP-Complete Complexity

Logical reasoning is far more computationally intensive than reflexes or arithmetic.⁴³ In an ideal scenario, a reasoning system would evaluate all possible logical inferences to determine the best conclusion, but this process quickly becomes computationally infeasible as the number of variables increases.

Humans, however, do not engage in exhaustive logical reasoning. Research by Daniel Kahneman and Amos Tversky suggests that humans instead use availability, representativeness, and anchoring heuristics that enable decision-making under constraints.⁴⁴ While this allows for fast, intuitive judgments, it does not guarantee optimal solutions, which is why cognitive biases and logical fallacies arise in human reasoning. Unlike brute-force algorithms, the human brain prioritizes finding acceptable, workable solutions rather than achieving perfect optimization.

Whether or not intelligence itself can truly be mechanized remains an open question. But what is clear is that time complexity—not just computability in principle—sets the boundary between theoretical possibility and practical feasibility.

The NP Condition

The decision problems that define the differences between the P and NP complexity classes offer an analogy to the kinds of problems that arise continually in daily life. Choosing a career path, designing a policy, and picking a college are all combinatorially large decisions with unclear constraints, shifting inputs, and qualitative objectives. The divide between knowing that something is good and knowing how to produce it is the signature of NP problems, and it pervades the structure of cognition itself.

⁴³ Many reasoning tasks involve solving satisfiability problems, which are NP-complete.

⁴⁴ Daniel Kahneman, Paul Slovic, and Amos Tversky, eds., *Judgment under Uncertainty: Heuristics and Biases* (Cambridge: Cambridge University Press, 1982).

Heuristics Over Optimization

If these mental shortcuts – heuristics – allow the brain to efficiently compute problems in a reasonable amount of time, then perhaps they are the key to understanding how people operate. Consider a real-world decision like choosing a graduate advisor. One could, in principle, evaluate every professor in every department across multiple countries, weigh their research, funding, personality, mentorship style, and departmental politics, and compute the optimal match. However, this approach would amount to a brute-force search over an exponentially large decision space, which is computationally infeasible in practice. This is not how students make choices. We do not calculate optimization functions over all consequences. Instead, we rely on recommendations, local knowledge, intuitive fit, and often revise as we go.

These strategies are not failures of rationality. They are *the cognitive equivalent of algorithmic approximation schemes*: fast, frugal, adaptive, and meaningful within bounded contexts. In this way, real-world cognition mirrors the structure of intractable problems. It is not that we fail to optimize; rather, optimization is structurally out of reach, and cognition is necessarily organized around heuristic approximation.

Gerd Gigerenzer, in his work on ecological rationality, argues that heuristics can outperform optimization when the environment matches the structure of the heuristic.⁴⁵ This is a foundational insight: intelligence is not the ability to solve all problems, but to solve the right kinds of problems, at the right scale, and in the right way.

If this view is right, then much of what we consider distinctively human – uncertainty, improvisation, error, forgiveness – is not a sign of our limits, but a reflection of the structure of the world itself. We are not imperfect computers. We are agents shaped by, and suited to, a world of NP problems.

The Complexity of Leadership

Building on the role of heuristics in cognition, social scientist Herbert A. Simon developed his *theory of bounded rationality* where he argues that people are rational agents and not utility-maximizers. We are agents who seek solutions that are “good enough” under conditions of uncertainty and limited cognitive resources.⁴⁶

⁴⁵ Peter M. Todd and Gerd Gigerenzer, *Ecological Rationality: Intelligence in the World* (New York: Oxford University Press, 2012).

⁴⁶ Herbert A. Simon, “Theories of Bounded Rationality,” in *Decision and Organization*, eds. C. B. McGuire and Roy Radner (Amsterdam: North-Holland Publishing, 1972), 161–176.

Simon's concept of bounded rationality laid the groundwork for computational organization science, a formalism developed in the 1990s to model how collections of interacting agents behave in complex organizational settings.⁴⁷ When groups of agents perform specific tasks, some individuals become overwhelmed by the density of interactions at the center of the network. As tasks and interactions grow in complexity, some agents assume more central positions within the network, developing a disproportionate number of connections with other agents.⁴⁸ These agents at the center of the network then exhibit behaviors that constitute leadership.⁴⁹

However, under the assumptions of bounded rationality, these highly connected agents are subject to *cognitive overload*, a state where the demands of the position exceeds one's ability to function successfully.⁵⁰ If left unchecked, this overload can contribute to a "complexity catastrophe" at the system level, where excessive interdependencies may lead to bottlenecks and severely degrade performance.⁵¹

In this context, leadership should not be centered around a static position of authority, but should assume a dynamic redistribution of decision-making and information flow throughout the network. This leads directly to the need for *shared leadership*.⁵² In complex environments, leadership must be distributed throughout the system in order to capitalize on the intelligence embedded within the network.⁵³

In practice, leadership roles must shift based on context and need. Authority should not be fixed in a hierarchy, nor overburdening top performers and formal leaders. Systems ought to be designed that support distributed-decision making. In environments where no single individual can process all relevant information or foresee all consequences, distributing leadership allows systems to remain responsive and adaptive.

⁴⁷ Kathleen M. Carley and Michael J. Prietula, eds., *Computational Organization Theory* (Hillsdale, NJ: Lawrence Erlbaum Associates, 1994).

⁴⁸ Albert-László Barabási, *Linked: The New Science of Networks* (Cambridge, MA: Perseus Publishing, 2002).

⁴⁹ Kathleen M. Carley and Yuqing Ren, "Tradeoffs between Performance and Adaptability for C3I Architectures" (paper presented at the Command and Control Research and Technology Symposium, Annapolis, MD, June 2001).

⁵⁰ James K. Hazy, "Toward a Theory of Leadership in Complex Systems: Computational Modeling Explorations," *Nonlinear Dynamics, Psychology, and Life Sciences* 12 (2008): 281–310.

⁵¹ Stuart A. Kauffman, *At Home in the Universe: The Search for the Laws of Self-Organization and Complexity* (New York: Oxford University Press, 1995).

⁵² Nicholas Clarke, "Model of Complexity Leadership Development," *Human Resource Development International* 16, no. 2 (2013): 135–150, <https://doi.org/10.1080/13678868.2012.756155>.

⁵³ Mary Uhl-Bien, Russ Marion, and Bill McKelvey, "Complexity Leadership Theory: Shifting Leadership from the Industrial Age to the Knowledge Era," *The Leadership Quarterly* 18, no. 4 (2007): 298–318.

Moreover, new knowledge arises through interactions between system members who generate shared understanding, identify tensions, and co-create solutions.⁵⁴ Knowledge creation becomes a social and local process, dependent on dynamic sense-making and the continual reinterpretation of complex information.

In complexity theory, a problem is intractable when no efficient algorithm can solve it within practical resource bounds. In everyday language, “intractable” describes problems—or people—that are difficult, messy, and resistant to control. This double meaning is not accidental and is especially relevant in terms of leadership. Both senses capture the conditions that leaders face regularly: environments and actors that cannot be solved, optimized, or neatly managed. When dealing with intractable systems or individuals, leaders must act under the assumption that complexity cannot be eliminated, only navigated.

Traditional leadership research has focused on improving the leader-follower relationship, emphasizing the traits, behaviors, or competencies of individual leaders. Yet the rapid social, economic, and technological shifts of the modern era have generated conditions that are increasingly complex, dynamic, and unstable.⁵⁵ These conditions place major constraints on conventional constructs of leadership, making the effectiveness of linear, cause-effect models increasingly untenable. Much like traditional approaches to computation, the traditional leadership approaches fail to account for the extent to which leadership depends on situated contexts, instead focusing on one correct “brute-force” method of leading.⁵⁶

In leadership, as in computation, we must recognize that the agents may face situations resembling NP problems, challenging the reductionist approaches that assume leadership and its impact within complex systems can be captured by simple, linear, cause-effect relationships.⁵⁷ Thus, complexity theory, and the recognition that many real-world leadership challenges mirror NP problems, enables the conditions under which adaptive responses can emerge.

Extending the NP Condition

Take the tasks that humans are uniquely good at, such as painting, writing poetry, and composing music. If there was a polynomial-time algorithm that could *recognize* great art or poetry, then *creating* masterpieces would itself be an NP task.⁵⁸ Importantly, the “NP-ness” of the world is

⁵⁴ Stuart A. Kauffman, *The Origins of Order* (New York: Oxford University Press, 1993).

⁵⁵ Hazy, “Toward a Theory of Leadership in Complex Systems.”

⁵⁶ Clarke, “Model of Complexity Leadership Development.”

⁵⁷ Ilya Prigogine, *The End of Certainty* (New York: Free Press, 1997).

⁵⁸ Aaronson, “Why Philosophers Should Care,” 15.

not confined to engineering or math. It appears wherever recognizing value is easier than producing it.

In ethics, we can judge a just outcome, but cannot compute a just policy from first principles. We recognize a just decision, like when a choice treats people fairly, distributes burdens appropriately, and balances competing values. But formulating a system that yields such decisions in general, let alone computing one on demand, is difficult as the complexity of human values resists reduction to any tractable optimization.

Similarly, in love and relationships we often recognize mutual compatibility—shared goals, trust, resonance—but cannot derive intimacy through computation. The matching problem of human connection is not reducible to personality variables, compatibility scores, or survey data. Relationships unfold historically, unpredictably, through feedback loops of vulnerability, misunderstanding, forgiveness, and time.

The same problem persists in art and aesthetics. We often know when something “works,” when a poem lands, or a painting moves us, but that recognition is not derived from a formal criteria. No function can map an idea to a masterpiece in polynomial time.

This difference between search (P) and verification (NP)—between knowing *what* and knowing *how*—is not merely technical. It maps onto something deep in human experience. The human condition is, fundamentally, an NP condition.

Conclusion

This paper has argued that complexity is an important lens in which to view intelligence. We live, think, and lead within an NP condition. The image of intelligence reduced to pre-fabricated fragments, like tools packed in pockets and assembled on demand, offers a prescient warning. In an age increasingly shaped by artificial intelligence, we risk mistaking storage for understanding, speed for reasoning, and replication for thought. Intractability is a defining feature of the world we inhabit. We must build systems, institutions, and forms of leadership that acknowledge complexity and act within its bounds.

References

“2136279841-1 Is the New Largest Known Prime Number,” *Great Internet Mersenne Prime Search*, Mersenne Research, Inc., last modified October 21, 2024, <https://www.mersenne.org/>.

Aaronson, Scott. “Why Philosophers Should Care About Computational Complexity.” *MIT Press*, 2012, p.5. <https://doi.org/https://doi.org/10.48550/arXiv.1108.1791>.

Assaad, Zena. “ChatGPT Just Passed the Turing Test. But That Doesn’t Mean AI Is Now as Smart as Humans.” *The Conversation*, April 8, 2025. <https://theconversation.com/chatgpt-just-passed-the-turing-test-but-that-doesnt-mean-ai-is-now-a-s-smart-as-humans-253946>.

Barabási, Albert-László. *Linked: The New Science of Networks*. Cambridge, MA: Perseus Publishing, 2002.

Bousso, Raphael. “Positive Vacuum Energy and the N-Bound.” *Journal of High Energy Physics* 2000, no. 11 (2000): 038. <https://arxiv.org/abs/hep-th/0010252>.

Carley, Kathleen M., and Michael J. Prietula, eds. *Computational Organization Theory*. Hillsdale, NJ: Lawrence Erlbaum Associates, 1994.

Carley, Kathleen M., and Yuqing Ren. “Tradeoffs between Performance and Adaptability for C3I Architectures.” Paper presented at the Command and Control Research and Technology Symposium, Annapolis, MD, June 2001.

Clarke, Nicholas. “Model of Complexity Leadership Development.” *Human Resource Development International* 16, no. 2 (2013): 135–150. <https://doi.org/10.1080/13678868.2012.756155>.

Cole, David. “The Chinese Room Argument.” In *The Stanford Encyclopedia of Philosophy*, Winter 2024 Edition. Edited by Edward N. Zalta and Uri Nodelman.

“Difference Between Exponential and Polynomial Complexities.” *GeeksforGeeks*. Sanchhaya Education Private Limited. Last modified August 6, 2024. <https://www.geeksforgeeks.org/difference-between-exponential-and-polynomial-complexities/>.

Gebicke-Haerter, Peter J. “The Computational Power of the Human Brain.” *Frontiers in Cellular Neuroscience* 17 (2023). <https://doi.org/10.3389/fncel.2023.1220030>.

Hameroff, Stuart, and Roger Penrose. “Consciousness in the Universe: A Review of the ‘Orch OR’ Theory.” *Physics of Life Reviews* 11, no. 1 (2014): 39–78. <https://doi.org/10.1016/j.plrev.2013.08.002>.

Hazy, James K. “Toward a Theory of Leadership in Complex Systems: Computational Modeling Explorations.” *Nonlinear Dynamics, Psychology, and Life Sciences* 12 (2008): 281–310.

Immerman, Neil. “Computability and Complexity.” In *The Stanford Encyclopedia of Philosophy*, Winter 2021 Edition. Edited by Edward N. Zalta.

Kahneman, Daniel, Paul Slovic, and Amos Tversky, eds. *Judgment under Uncertainty: Heuristics and Biases*. Cambridge: Cambridge University Press, 1982.

Kauffman, Stuart A. *At Home in the Universe: The Search for the Laws of Self-Organization and Complexity*. New York: Oxford University Press, 1995.

———. *The Origins of Order*. New York: Oxford University Press, 1993.

“Mersenne Prime: 2136279841-1 Is the New Largest Known Prime Number.” *Great Internet Mersenne Prime Search*. Mersenne Research, Inc. Last modified October 21, 2024. <https://www.mersenne.org/>.

“Multiplication: Definition, Symbol, Table.” *GeeksforGeeks*. Sanchhaya Education Private Limited. Last modified April 16, 2025. <https://www.geeksforgeeks.org/multiplication/>.

Nadis, Steve. “New Breakthrough Brings Matrix Multiplication Closer to Ideal.” *Quanta Magazine*, Simons Foundation, March 7, 2024. <https://www.quantamagazine.org/new-breakthrough-brings-matrix-multiplication-closer-to-ideal-20240307/>.

Nagarajan, Naveen, and Charles F. Stevens. “How Does the Speed of Thought Compare for Brains and Digital Computers?” *Current Biology* 18, no. 17 (2008): R756–R758. <https://doi.org/10.1016/j.cub.2008.06.043>.

Prigogine, Ilya. *The End of Certainty*. New York: Free Press, 1997.

Raatikainen, Panu. “Gödel’s Incompleteness Theorems.” In *The Stanford Encyclopedia of Philosophy*, Spring 2022 Edition. Edited by Edward N. Zalta.

Rescorla, Michael. “The Computational Theory of Mind.” In *The Stanford Encyclopedia of Philosophy*, Winter 2024 Edition. Edited by Edward N. Zalta and Uri Nodelman.

Simon, Herbert A. “Theories of Bounded Rationality.” In *Decision and Organization*, edited by C. B. McGuire and Roy Radner, 161–176. Amsterdam: North-Holland Publishing, 1972.

Sipser, Michael. “The History and Status of the P Versus NP Question.” In *Proceedings of the ACM Symposium on Theory of Computing*, 603–618. New York: ACM, 1992.

Todd, Peter M., and Gerd Gigerenzer. *Ecological Rationality: Intelligence in the World*. New York: Oxford University Press, 2012.

Turing, Alan M. “Computing Machinery and Intelligence.” *Mind* 59, no. 236 (1950): 433–460.

——— “On Computable Numbers, with an Application to the Entscheidungsproblem.” *Proceedings of the London Mathematical Society* 42, no. 1 (1936): 230–265.
<https://doi.org/10.1112/plms/s2-42.1.230>.

“Turing Test.” *GeeksforGeeks*. Sanchhaya Education Private Limited. Last modified September 16, 2024. <https://www.geeksforgeeks.org/turing-test-artificial-intelligence/>.

Uhl-Bien, Mary, Russ Marion, and Bill McKelvey. “Complexity Leadership Theory: Shifting Leadership from the Industrial Age to the Knowledge Era.” *The Leadership Quarterly* 18, no. 4 (2007): 298–318.

Williams, Rachel Anne. “Searle’s Chinese Room Thought Experiment: A Twist.” *Medium*, April 15, 2018.
<https://medium.com/@transphilosophr/searles-chinese-room-thought-experiment-a-twist-c7eb28f65e6c>.

Yu, Jessica. “Traveling Salesman Problem.” *Computational Optimization Open Textbook – Optimization Wiki*, Cornell University, Spring 2014.
https://optimization.cbe.cornell.edu/index.php?title=Traveling_salesman_problem.